模块 7：工程实战 — 两大工具深度使用

模块概述

> **核心问题：如何在真实工程中高效使用 AI 编程 Agent?**

前面六个模块我们拆解了 AI 编程 Agent 的原理与策略。现在进入实战——把理论落地到日常开发中。

本模块聚焦两款成熟的终端 AI 编程工具：**Claude Code** 和 **OpenCode**。它们代表了两种典型路线：

维度	Claude Code	OpenCode
模型绑定	Claude 模型（Anthropic 原生）	多模型支持（OpenAI / Anthropic / 本地模型）
架构模式	单 Agent + Agent Teams	单 Agent
代码感知	工具链（Glob/Grep/Read）	LSP 集成 + 工具链
扩展性	MCP Server / 自定义 Agent	自定义工具 / Provider
界面	CLI + IDE 集成	TUI（Terminal UI）

7.1 Claude Code 工程全貌

安装与初始化

Claude Code 是 Anthropic 官方的 CLI 工具，安装方式：

```
# 通□□npm 安装
npm install -g @anthropic-ai/claude-code

# □□安装
claude --version

# 首次□行，完成□□
claude
```

```
# 在目中初始化配置
claude /init
```

`/init` 命令会在项目根目录生成一个 `CLAUDE.md` 文件，这是 Claude Code 的核心配置之一。

CLAUDE.md：项目级指令系统

CLAUDE.md 是 Claude Code 最独特的设计之一。它不是普通的文档——**它是嵌入到 Agent 系统提示中的项目级指令**。每次 Claude Code 启动时，都会读取 CLAUDE.md 并将其内容作为上下文注入。

三级继承体系

```
~/.claude/CLAUDE.md              ← 用户（全局默偏好）
  │
  ▼
目根目/CLAUDE.md                 ← 目（目定与范）
  │
  ▼
目根目/src/CLAUDE.md             ← 目（子模特定指令）
```

三级指令会被合并（merge），优先级从高到低：目录级 > 项目级 > 用户级。这意味着你可以在全局设定通用偏好，然后在每个项目中覆盖特定规则。

CLAUDE.md 应该写什么

一个高效的 CLAUDE.md 模板：

```
# CLAUDE.md

## 目概述
是一个基于 Next.js 14 的商平台，使用 TypeScript + Prisma + PostgreSQL。

## 代范
- 使用 TypeScript strict mode
- 函数命名使用 camelCase，类型命名使用 PascalCase
- 件使用函数式件 + hooks，不使用 class 件
- 所有 API 路由在 app/api/ 目下
- 使用 Zod 做行类型校

## 常用命令
- `pnpm dev` — 开服器
```

```
- `pnpm test` — 行 (vitest)
- `pnpm test:e2e` — 行端到端 (Playwright)
- `pnpm lint` — ESLint
- `pnpm db:migrate` — 行数据迁移

## 架明
- app/ — Next.js App Router 面与路由
- lib/ — 共享工具函数和
- components/ — React 件（按功能模划分）
- prisma/ — 数据schema 和迁移文件

## 提交范
- 使用 Conventional Commits 格式：feat/fix/chore/docs
- PR 描述必包含更摘要和明
```

关键原则：CLAUDE.md 是给 Agent 看的，不是给人看的。要写得**明确、可执行**，避免模糊描述。

settings.json：权限与行为配置

`settings.json` 控制 Claude Code 的运行时行为，分为全局和项目级两层：

```
~/.claude/settings.json            ← 全局配置
目根目/.claude/settings.json    ← 目配置
```

典型配置示例：

```json
{
  "permissions": {
    "allow": [
      "Bash(npm run test)",
      "Bash(npm run lint)",
      "Bash(git status)",
      "Bash(git diff)",
      "Bash(git log)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'File being edited: $CLAUDE_FILE_PATH'"
```

```
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Command completed'"
          }
        ]
      }
    ]
  }
}
```

完整工具链

Claude Code 内置的工具集直接映射了 Agent 的感知-规划-执行循环:

```
感知□ (Grounding)
├── Read        — □取文件内容
├── Glob        — 按 pattern 搜索文件路径 (如 **/*.tsx)
├── Grep        — 按正□搜索文件内容 (基于 ripgrep)
├── WebFetch    — □取并分析网□内容
└── WebSearch   — 搜索网□信息

□行□ (Action)
├── Edit        — 精确的字符串替□□□
├── Write       — □建或完整覆写文件
├── Bash        — □行 shell 命令
└── NotebookEdit — □□□Jupyter notebook

□作□ (Collaboration)
├── Agent       — 生成子 Agent □理子任□
├── SendMessage — Agent □通信
├── TaskCreate  — □建任□
├── TaskUpdate  — 更新任□状□
└── TaskList    — 列出所有任□

□助□
├── AskUserQuestion — 向用□提□以□取澄清
└── EnterPlanMode   — □入□划模式, 先制定方案再□行
```

常用 Slash 命令

命令	用途
`/init`	初始化项目 CLAUDE.md
`/compact`	压缩对话历史，释放上下文空间
`/model`	切换模型（如 opus → sonnet 节省成本）
`/help`	查看帮助信息
`/clear`	清空当前对话
`/cost`	查看当前会话的 token 消耗和费用

IDE 集成

Claude Code 深度集成到 VS Code（官方扩展，支持选中代码直接发送给 Agent、终端面板内嵌交互界面）和 JetBrains（IntelliJ、WebStorm、PyCharm 等，通过插件支持）。

7.2 Agent Teams

核心动机与团队创建

单 Agent 面临上下文窗口限制、串行执行瓶颈和关注点发散三个问题。Agent Teams 通过**分工并行**解决——每个子 Agent 拥有独立上下文，专注单一任务，通过任务系统协调。

```
TeamCreate({
  team_name: "feature-auth",
  description: "⍰⍰用⍰⍰⍰功能"
})
→ 生成 ~/.claude/teams/feature-auth/config.json（⍰⍰配置）
→ 生成 ~/.claude/tasks/feature-auth/（任⍰列表）
```

任务管理：TaskCreate / TaskUpdate / TaskList

任务系统是 Agent Teams 的协调中枢：

```
# □建任□
TaskCreate({
  subject: "□□□JWT □□中□件",
  description: "在 lib/auth/middleware.ts 中□□□JWT □□...",
  activeForm: "Implementing JWT middleware"
})

TaskCreate({
  subject: "□写□□□API 路由",
  description: "在 app/api/auth/ 下□建 login/register/logout...",
  activeForm: "Creating auth API routes"
})

TaskCreate({
  subject: "□写□□功能的□□",
  description: "□□JWT 中□件和 API 路由□写□元□□...",
  activeForm: "Writing auth tests"
})
```

任务支持**依赖关系**——通过 `addBlockedBy` 确保执行顺序：

```
# □□任□依□于□□任□
TaskUpdate({
  taskId: "3",
  addBlockedBy: ["1", "2"]    ← □□必□在中□件和 API □□后才能开始
})
```

生成子 Agent

使用 `Agent` 工具生成具有不同职责的 Teammate：

```
# 生成后端□□□Agent
Agent({
  name: "backend-dev",
  subagent_type: "general-purpose",
  team_name: "feature-auth",
  prompt: "你□□□□后端□□□□。□从 TaskList □取你的任□。",
  run_in_background: true
})

# 生成□□□写 Agent
Agent({
  name: "test-writer",
  subagent_type: "general-purpose",
  team_name: "feature-auth",
  prompt: "你□□□写□□。□从 TaskList □取你的任□。",
  run_in_background: true
})

# 生成代□□□□Agent
Agent({
```

```
  name: "reviewer",
  subagent_type: "general-purpose",
  team_name: "feature-auth",
  prompt: "你□□□□其他 Agent 提交的代□。□从 TaskList □取你的任□。",
  run_in_background: true
})
```

关键参数:

参数	说明
`name`	Agent 名称，用于消息通信和任务分配
`subagent_type`	Agent 类型：`general-purpose`（全能）、`Explore`（只读探索）、`Plan`（规划）
`team_name`	所属团队名称
`run_in_background`	是否后台运行（`true` 则不阻塞主 Agent）
`isolation: "worktree"`	在 Git worktree 中运行，避免文件冲突

Agent 间通信

通过 `SendMessage` 工具实现 Agent 间通信，支持三种类型:

```
# 点□点消息
SendMessage({ type: "message", recipient: "backend-dev",
  content: "JWT 中□件需要支持 refresh token", summary: "JWT refresh token req" })

# 广播（慎用，□□所有 Agent）
SendMessage({ type: "broadcast",
  content: "API base URL 已改□ /api/v2", summary: "API URL changed" })

# 关□□Agent
SendMessage({ type: "shutdown_request", recipient: "test-writer",
  content: "任□完成" })
```

实际工作流示例

一个完整的多 Agent 功能开发流程:

```
Team Lead（主 Agent）
│
├── 1. 分析需求，□建□□和任□
```

```
│     TeamCreate("feature-search")
│     TaskCreate("□□搜索 API")          → Task #1
│     TaskCreate("□□搜索 UI □件")       → Task #2
│     TaskCreate("□写搜索功能□□")       → Task #3 (blocked by #1, #2)
│     TaskCreate("更新文档")              → Task #4 (blocked by #3)
│
├── 2. 生成子 Agent 并分配任□
│     Agent("api-dev")     → 分配 Task #1
│     Agent("ui-dev")      → 分配 Task #2
│      (Task #3, #4 □□□blocked)
│
├── 3. 子 Agent 并行工作
│     api-dev: □代□ → □□□API → □□□Task #1 完成
│     ui-dev:  □代□ → □□□件 → □□□Task #2 完成
│
├── 4. Task #3 解除阻塞
│     Agent("tester")      → 分配 Task #3
│     tester: □写□□ → □行□□ → 修复失□ → □□完成
│
├── 5. Task #4 解除阻塞
│     Team Lead 自己完成文档更新
│
└── 6. 清理
      SendMessage(shutdown_request → 所有子 Agent)
      TeamDelete()
```

最佳实践：只读任务用 `Explore` Agent（更低成本）；可能有文件冲突的 Agent 用 `isolation: "worktree"`；任务粒度不宜过细（至少数个文件的改动）；团队规模控制在 3-5 个 Agent。

7.3 OpenCode 工程全貌

安装与基本配置

OpenCode 是一个开源的终端 AI 编码助手，特点是多模型支持和 TUI 界面。

```
# 通□□Go 安装
go install github.com/opencode-ai/opencode@latest

# 或通□□npm
npm install -g opencode-ai

# 在□目目□中□□
cd your-project
opencode
```

多模型配置

OpenCode 的核心优势是灵活的多模型配置。配置文件位于项目根目录的 `opencode.json`：

```json
{
  "providers": {
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}",
      "disabled": false
    },
    "openai": {
      "apiKey": "${OPENAI_API_KEY}",
      "disabled": false
    },
    "ollama": {
      "baseURL": "http://localhost:11434",
      "disabled": false
    }
  },
  "models": {
    "claude-sonnet": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-6",
      "maxTokens": 16384
    },
    "gpt-4o": {
      "provider": "openai",
      "model": "gpt-4o",
      "maxTokens": 16384
    },
    "deepseek-local": {
      "provider": "ollama",
      "model": "deepseek-coder-v2:latest",
      "maxTokens": 8192
    }
  }
}
```

多模型策略：复杂架构设计用 Opus/GPT-4o（深度推理）、日常编码用 Sonnet/GPT-4o-mini（速度与质量平衡）、敏感代码用本地模型（数据不出网）、批量处理用低成本模型。

LSP 集成

OpenCode 的一大差异化特性是 **LSP（Language Server Protocol）集成**。这使得 Agent 不仅能做文本级的代码搜索，还能理解代码的语义结构：

```
□□□Agent 感知:
  Grep("handleAuth") → 找到所有包含 "handleAuth" 的行
  ❌ 无法区分：定□□vs 引用 vs 注□中的提及

LSP 增强感知:
  GoToDefinition("handleAuth") → 精确跳□到函数定□
  FindReferences("handleAuth") → 找到所有□用点
  GetDiagnostics() → □取□□□□和警告
  ✅□□□□别的代□理解
```

LSP 支持的能力：

LSP 功能	Agent 收益
Go to Definition	精确定位函数/类/变量定义
Find References	分析变更的影响范围
Diagnostics	实时获取类型错误和语法错误
Code Actions	获取 IDE 级别的修复建议
Hover Information	获取类型签名和文档

Session 管理

OpenCode 支持 session 的持久化与恢复：

```
# 列出□史 session
opencode sessions list

# 恢复之前的 session
opencode sessions resume <session-id>
```

TUI 特性

OpenCode 的 TUI 提供了实时状态可视化（可见 Agent 正在执行的工具）、多面板布局（对话/差异/终端并列）、快捷键操作和 Session 快速切换等增强体验。

7.4 真实工作流

场景 1：Bug 修复全流程

一个典型的 Agent 驱动 bug 修复流程：

用户报告：“登录接口偶尔返回 500 错误，日志中有 `TypeError: Cannot read property 'email' of null`”

```
用⍰⍰入:
> 修复登⍰接口的 500 ⍰⍰。⍰⍰日志：
> TypeError: Cannot read property 'email' of null
> 出⍰在 app/api/auth/login/route.ts

Agent ⍰行⍰程:

Step 1: ⍰取相关代⍰
────────────────────
工具: Read("app/api/auth/login/route.ts")

// route.ts
export async function POST(req: Request) {
  const body = await req.json()
  const user = await db.user.findUnique({
    where: { email: body.email }
  })
  const isValid = await bcrypt.compare(body.password, user.passwordHash)
  //                                                    ^^^^ 当 user ⍰⍰null ⍰崩⍰
  ...
}

Step 2: 分析根因
────────────────────
Agent ⍰别出: 当 email 不存在⍰, user ⍰⍰null,
直接⍰⍰⍰user.passwordHash ⍰致 TypeError

Step 3: 搜索类似模式
────────────────────
工具: Grep("findUnique")
→ ⍰⍰其他 3 个文件有类似的未⍰⍰⍰null 的模式

Step 4: 修复代⍰
────────────────────
工具: Edit("app/api/auth/login/route.ts")
old_string: |
  const isValid = await bcrypt.compare(body.password, user.passwordHash)
new_string: |
  if (!user) {
    return Response.json({ error: "Invalid credentials" }, { status: 401 })
  }
  const isValid = await bcrypt.compare(body.password, user.passwordHash)
```

```
Step 5: 行
────────────────────
工具: Bash("pnpm test app/api/auth")
✓ 15 tests passed

Step 6: 提交
────────────────────
工具: Bash("git add app/api/auth/login/route.ts")
工具: Bash("git commit -m 'fix: handle null user in login route'")
```

场景 2：功能开发全流程

需求：“给用户资料页添加头像上传功能（JPG/PNG，最大 5MB，上传到 S3）”

```
Agent 行程:

Step 1: 入划模式 → EnterPlanMode()
────────────────────
Agent 探索代:
- Glob("**/profile/**") → 找到料相关文件
- Read("app/profile/page.tsx") → 理解有面
- Read("lib/db/schema.prisma") → 确User 模型
- Grep("S3|upload|aws") → 找有的 S3 配置

出划:
1. 更新 Prisma schema，添加 avatarUrl 字段
2. 建 S3 上工具函数 lib/storage/s3.ts
3. 建 API 路由 app/api/user/avatar/route.ts
4. 在料添加上件
5. 写

Step 2-6: 用批准后，Agent 按划逐步行

Step 7: 建 PR
────────────────────
Bash("git checkout -b feat/avatar-upload")
Bash("git add -A && git commit -m 'feat: add avatar upload'")
Bash("gh pr create --title 'feat: add avatar upload' --body '...'")
```

场景 3：代码审查辅助

```
用入: > PR #142

Agent 行:
1. Bash("gh pr view 142 --json title,body,files") + Bash("gh pr diff 142")
2. 逐文件：正确性、理、类型安全、安全漏洞、性能影响
3. 出化意:

## PR #142 意
```

```
### ⍰⍰
1. lib/api/handler.ts:45 — 未⍰理 null 返回⍰，可能⍰致⍰行⍰⍰⍰
2. app/api/data/route.ts:23 — SQL 拼接存在注入⍰⍰，建⍰参数化⍰⍰
3. lib/cache.ts:12 — ⍰存未⍰置⍰期⍰⍰，可能⍰致内存泄漏
### ⍰体⍰价
功能⍰⍰正确，但需要修复安全⍰⍰后再合入。
```

7.5 大型代码库策略

核心矛盾

Agent 的上下文窗口（即使 200K tokens）无法装下一个数十万行的 Monorepo，但修复一个 bug 可能需要理解跨越多个 package 的调用链。以下四个策略帮助 Agent 在大型代码库中高效工作。

策略 1：CLAUDE.md 充当架构地图

在大型项目的 CLAUDE.md 中提供**架构导航信息**：

```
# CLAUDE.md（大型⍰目版）

## 架⍰概⍰
本⍰目采用 Monorepo ⍰⍰（pnpm workspace）。

## 模⍰依⍰关系
packages/shared → 被所有其他 package 引用
packages/api → 依⍰⍰shared，提供 REST + GraphQL API
packages/web → 依⍰⍰shared + api 的类型定⍰
packages/mobile → 依⍰⍰shared，使用 React Native

## 关⍰入口点
- API 服⍰入口：packages/api/src/index.ts
- Web ⍰用入口：packages/web/app/layout.tsx
- 共享类型定⍰：packages/shared/src/types/index.ts
- 数据⍰⍰Schema：packages/api/prisma/schema.prisma

## 模⍰⍰⍰速⍰
| 目⍰⍰ | ⍰⍰⍰⍰ | ⍰⍰⍰⍰⍰⍰ |
|---------|---------|---------|
| packages/api/src/routes/ | API 路由定⍰⍰ | 后端⍰⍰ |
| packages/api/src/services/ | ⍰⍰⍰⍰⍰⍰⍰ | 后端⍰⍰ |
| packages/web/app/ | ⍰⍰面与路由 | 前端⍰⍰ |
| packages/web/components/ | UI ⍰件 | 前端⍰⍰ |
| packages/shared/src/utils/ | 工具函数 | 平台⍰⍰ |
```

这份信息让 Agent 在开始探索前就知道**去哪里找**——相当于给它一张地图。

策略 2：定向探索（Glob + Grep 组合拳）

不要让 Agent 漫无目的地浏览代码。用精准的搜索缩小范围：

```
# 第一步：找到相关文件
Glob("packages/api/src/**/*auth*")
→ 找到 5 个⍰⍰相关文件

# 第二步：在⍰些文件中搜索关⍰⍰⍰
Grep("validateToken", path="packages/api/src/")
→ 精确定位 token ⍰⍰⍰⍰

# 第三步：追踪⍰用⍰
Grep("import.*from.*auth", path="packages/")
→ 找到所有引用⍰⍰模⍰的文件
```

核心原则：**先缩小范围，再深入阅读**。在大型代码库中，Agent 的每一次 `Read` 都应该是有目的的，而不是”读一读看看”。

策略 3：分层感知（自顶向下）

```
⍰次 1: ⍰目⍰⍰（Glob + ls）
    → 了解 package 划分和目⍰⍰⍰
    → 成本：极低（只看文件名）

⍰次 2: 模⍰接口（Read 入口文件和类型定⍰）
    → 了解模⍰⍰的 API 契⍰
    → 成本：中等（只⍰关⍰文件）

⍰次 3: 具体⍰⍰（Read + Grep 定向搜索）
    → 深入特定函数的⍰⍰⍰⍰
    → 成本：⍰高（⍰大量代⍰）

⍰次 4: ⍰行⍰行⍰（Bash ⍰行⍰⍰/⍰⍰）
    → ⍰⍰理解是否正确
    → 成本：最高（⍰⍰⍰行代⍰）
```

最佳实践：从层次 1 开始，逐层深入，而不是上来就读所有文件。每一层都应该为下一层的搜索提供方向。

策略 4：.claudeignore 限制范围

类似 `.gitignore`，`.claudeignore` 告诉 Claude Code 忽略特定文件和目录：

```
# .claudeignore

# 构建产物
dist/
build/
.next/
node_modules/

# 生成代码
packages/api/src/generated/
packages/web/src/__generated__/

# 大型数据文件
*.csv
*.json.gz
data/fixtures/

# 不相关的包
packages/legacy-app/
packages/deprecated-*
```

效果：减少搜索空间、降低上下文污染（生成代码不干扰 Agent 理解）、加快 Glob/Grep 速度。

7.6 何时不该用代理

Agent 不是银弹

理解 Agent 的局限性，与理解它的能力同样重要。盲目使用 Agent 可能导致：

- **时间浪费**：Agent 反复试错的时间超过了人工编码的时间
- **质量隐患**：Agent 生成的代码”看起来对”但存在细微问题
- **过度工程**：Agent 倾向于生成更多代码，而不是更简洁的解决方案

Agent 挣扎的任务类型

1. 高度创造性的设计

```
❌ "□□一个□新的用□交互模式"
❌ "□□个□品□□一套独特的□□系□"

Agent 擅□的是在已知模式中工作。开□性□□需要人类的□美判断和□造力。
```

2. 性能优化的”最后一英里”

```
❌ "把□个函数的□行□□从 50ms □化到 5ms"

Agent 能做出通用的□化建□（使用□存、减少循□嵌套等），
但深□的性能□化需要□□行□行□的深度理解——
CPU cache 命中率、内存分配模式、GC □力等。
```

3. 安全关键代码

```
❌ "□□加密密□□□□□"
❌ "□写支付□理的核心流程"

安全代□中的一个□微□□可能□致灾□性后果。
Agent 生成的代□可能通□所有□□，但存在 timing attack、
□信道泄漏等人类安全□家才能□□的□□。
```

4. 复杂的状态机和并发逻辑

```
❌ "□□分布式□的□租机制"
❌ "修复□个 race condition"

并□□□往往无法通□□□复□，需要□□行□序的深刻理解。
Agent 擅□□理确定性□□，但□非确定性的并□□景缺乏可靠推理能力。
```

不值得使用 Agent 的场景

有些任务太简单，使用 Agent 的开销（启动时间、API 费用、上下文加载）反而大于直接手写：

```
不□得用 Agent:
- 修改一个配置常量
- 重命名一个□量（IDE 重□功能更快）
- 添加一行日志□出
- □□的 typo 修复
```

```
□得用 Agent:
- 跨多个文件的重□
- 理解不熟悉的代□□
- □写重复性的模板代□
- 从 issue 到 PR 的完整流程
```

“最后 10%” 问题

Agent 能快速完成 90% 的工作（正确结构、通过基本测试、符合规范），但**最后 10% 需要人类介入**——边界条件的微妙处理、代码风格一致性、用户体验细节、业务规则特殊情况。最高效的工作模式是**混合模式**。

推荐的混合工作流

```
□段 1: Agent □□（适合 Agent）
├── 代□□探索和理解
├── 生成初始□□
├── □写□□框架
└── 生成文档骨架

□段 2: 人机□作（两者互□）
├── Agent 生成代□，人类□□
├── 人类指出方向，Agent □行
├── Agent □理重复性修改，人类□理□造性部分
└── 人类写核心□□，Agent 写□□

□段 3: 人类主□（适合人类）
├── 最□代□□□
├── 性能□□
├── 安全□□
└── □品体□打磨
```

实操环节

练习 1：使用 Claude Code Agent Teams 完成多文件功能开发

目标：使用 Agent Teams 为一个 Express 应用添加用户注册和登录功能。

步骤:

1. 克隆模板项目并启动 Claude Code:
   `git clone <template-repo-url> && cd agent-teams-lab && claude`

2. 使用自然语言描述任务:

```
> 使用 Agent Teams ⍰⍰用⍰⍰⍰功能。需要：
> 1. 用⍰注册 API (POST /api/register)
> 2. 用⍰登⍰⍰API (POST /api/login)
> 3. JWT token ⍰⍰中⍰件
> 4. 完整的⍰元⍰⍰
> ⍰⍰建一个 3 人⍰⍰来并行开⍰。
```

1. 观察并记录：Agent 如何创建团队和分配任务、子 Agent 之间如何协调、任务依赖如何影响执行顺序

2. 运行 `npm test` 和 `curl -X POST http://localhost:3000/api/register -H "Content-Type: application/json" -d '{"email":"test@example.com","password":"secure123"}'` 验证结果

练习 2：配置 OpenCode 多模型，对比同一任务的输出

目标：配置 OpenCode 接入至少 2 个不同的模型 Provider，对同一代码任务对比不同模型的输出。

步骤:

1. **配置 opencode.json**

```json
{
  "providers": {
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}"
    },
    "openai": {
      "apiKey": "${OPENAI_API_KEY}"
    }
  },
  "models": {
    "claude-sonnet": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-6",
      "maxTokens": 16384
    },
    "gpt-4o": {
      "provider": "openai",
```

```
      "model": "gpt-4o",
      "maxTokens": 16384
    }
  }
}
```

1. **准备测试任务**

向两个模型分别提交以下任务：

```
重□下面□段代□，提取重复□□，添加□□□理：
[准□一段有重复□□和缺少□□□理的代□]
```

1. **对比分析**

记录并对比：

对比维度	Claude Sonnet	GPT-4o
代码结构		
错误处理策略		
命名风格		
是否过度工程		
执行速度		
首次正确率		

本模块小结

核心要点回顾

```
Claude Code 工程体系
├── CLAUDE.md — □目□□Agent 指令（三□□承）
├── settings.json — 权限与 Hooks 配置
├── 完整工具□ — Read/Edit/Write/Glob/Grep/Bash/Agent
└── Agent Teams — 多 Agent 分工□作

OpenCode 差异化□□
├── 多模型灵活接入 — 同一工具切□不同模型
├── LSP 集成 — □□□代□感知
├── TUI 界面 — 更丰富的□端交互体□
└── Session 管理 — □任□的上下文持久化

大型代□□策略
```

```
├── CLAUDE.md 架□地□ — □□Agent 方向感
├── 定向搜索 — Glob + Grep □合拳
├── 分□感知 — 自□向下逐□深入
└── .claudeignore — 减少噪音

Agent 使用□界
├── 擅□ — 探索、模板代□、跨文件修改、流程自□化
├── □扎 — □造性□□、性能□化、安全代□、并□□□
└── 最佳□践 — 混合模式，Agent 做 90%，人类把控 10%
```

思考题

1. **CLAUDE.md 设计**：如果你要为自己当前工作的项目编写 CLAUDE.md，你会包含哪些信息？为什么？试着写出一个初稿，然后评估它是否足够”可执行”。

2. **Agent Teams 的权衡**：在什么样的任务中，使用 3 个 Agent 并行会比单个 Agent 串行更快？考虑任务分解的开销、文件冲突的风险、以及上下文切换的成本。

3. **多模型策略**：假设你的团队同时在开发前端 UI 和后端 API。你会如何配置多模型策略来优化成本和质量？不同的任务类型应该使用什么模型？

4. **Agent 边界判断**：列举你最近遇到的 5 个编程任务，分别判断它们是否适合使用 Agent。对于”适合”的任务，你会使用什么工作流？对于”不适合”的任务，为什么？

5. **大型代码库挑战**：假设你加入一个拥有 50 万行代码的 Monorepo 项目。你第一天如何利用 Agent 快速建立对项目的整体理解？描述你的具体探索步骤。